



The Building Block Economy

The most effective way to build software and get massive adoption is no longer high quality mainline apps but via building blocks that enable and encourage others to build quantity over quality.¹

- Ghostty in **18 months**: one million daily macOS update checks.
- libghostty in **2 months**: multiple millions of daily users.²

Similar growth trajectories can be seen in other "building block" technologies: Pi Mono, Next.js, Tailwind, etc.

Experiencing this firsthand as well as witnessing it in other ecosystems has fundamentally shifted how I view the practice of product and software development today, regardless of commercial vs non-commercial goals.

This article was written by hand, without the assistance of AI. I love and use AI abundantly, but I draw the line personally at content like this. I want my personal blog to reflect my genuine thoughts and feelings.

Imports Are Up

I use the term "building block" to describe how they're being used because they're being assembled today in a very different way than former decades. I don't use the term "library" or "framework" because it extends even up to

"applications" (e.g. the Ghostty GUI app has more forks than ever before with customized patches on top, which is awesome).

The factory of today is agentic³. I say that as objective truth, regardless of what your feelings about it are. You can argue that 99% of the stuff coming out of these factories is total crap, but you can't argue the sheer quantity of stuff coming out. The numbers are everywhere spanning tech stacks and industries and they're undeniable.

AI is okay at building everything from scratch, but it is *really good* at gluing together high quality, well documented, and proven components. And, AI prefers to do this when it can unless explicitly prompted otherwise. This is the "building block" nature of software today: we're more than ever before grabbing off the shelf components and gluing them together.

Humans, of course, have always done this as well. For my entire career, human software developers have preferred to build on top of proven primitives. But the natural barrier to entry of understanding the component pieces well enough to even slap them together was high enough to limit the ecosystem. This barrier is now gone.

Exports Are Up

Coming out of these factories is of course software. So much software.

There are negatives to this. I think the negatives are obvious enough that I'm not going to dedicate much time to them, but I want to recognize they exist: security vulnerabilities, instability, a general lack of understanding about how load-bearing systems might work.

But there are a huge amount of positives:

- **The quality bar is lower.** A mainline application used by a large cross-section of users has to weigh every

feature against every other feature: how do they interact? does it make sense for the long term vision? can I maintain this for millions of users? A factory artifact targeting one to hundreds of users doesn't need to care about this. You can ship faster and looser, as a result.

- **The awareness is greater.** A mainline application can't do everything. It usually optimizes for the use cases that the most users need and use. A factory artifact can optimize for a tiny cross-section of users, and these users gain awareness of the building block as a result. I'm seeing this hugely in Ghostty, as very niche communities are getting terminals.
- **The maintenance burden is lower.** It is easier than ever to say "no thank you" to feature requests, because you're offering a key part of the means to production. My challenges with slop requests are very public, to the point I made a "no machine", but I'm also feeling less and less bad every day about saying "no."
- **R&D is outsourced.** It is so much easier now as a maintainer to look at what others are doing, see working proof of concepts, and decide what you want to bring back to mainline. There's way less talk and way more walk. And while others walk you can cherry pick the best ideas (this is fair: you're giving away a building block and they're giving away their ideas).

The Impact

This is changing how I view software and product development.

I'm much more purposeful about creating building blocks and encouraging applications or forks on top of that. I think this is resulting in a happier community, a larger community, and ultimately better mainline software.

High-quality applications aren't disappearing. And high-quality applications produced by the developers of the building block aren't disappearing. For most software categories, I think there will always be a majority group that doesn't want personalized slop software and wants a polished, well maintained, and well supported application.

Instead, I think the mainline application is becoming more stable and more purposeful in its feature set thanks to the building block economy. The stability comes from a much larger and diverse user group. The feature set comes from the massive ecosystem of outsourced R&D. The mainline application is still measured, high quality, and well maintained, but it's for a specific group.

The Elephant in the Room: Commercialization

The obvious question that follows is what this can mean for commercialization. Closed source, commercial software appears to be at a massive disadvantage. And it is.

Agents will more readily pick open and free software over closed and commercial. At the time of writing this article, this is an objective truth. Independent research labs running experiments on popular models have found repeatedly that under diverse circumstances, models pick open and free alternatives over commercial. So far.

But, I don't have a concrete answer here, because unlike product and software development, I'm not directly building a commercializable product right now. I have thoughts, and as with all hard things, I think the answer is nuanced. But, I don't want to give the illusion of talking authoritatively about this so I'm going to avoid it. When I walk the walk and learn more, I'll share more.

I'm once again simply acknowledging that this challenge obviously exists.

The Shift Has Happened

We have to accept that building blocks and software factories rule everything around us and accept and internalize the consequences of that.

We can choose to run the other direction and create [enclaves where we fight against it](#). Or we can choose to [submit ourselves completely to the chaos](#). People who know me know I'm far less extreme in my actions and carry different opinions depending on context.

The point is the shift has already happened. We're living in it.

-
1. The building blocks themselves must usually be high quality, robust, and well documented, though. ↩
 2. Getting exact numbers is obviously hard since Ghostty has no real tracking. We can see aggregate update file checks for macOS. We have no visibility at all into Linux. libghostty has no tracking but tools integrating libghostty might and have shared aggregates with us. ↩
 3. I use the term "agent" here to simply mean a large language model with access to tools in a loop. People sometimes think I'm trying to be hype or cute by using marketing terms, but I am using this as a specific, well accepted definition. ↩